

Conception — Ouverture de l'application à plusieurs foyers

Document de conception. Version 1.0 — 2026-08-24. On code **phase par phase** dans cet ordre (P0 → P5). Chaque phase est livrable seule.

1. Objectif

Passer d'une application installée pour **un** foyer à une application **déployée une fois, exploitée par N foyers**, chacun avec ses propres données, son propre nom, son propre logo — sans qu'aucune ligne de code ne soit spécifique à l'un d'eux.

Deux exigences distinctes, souvent confondues :

1. **Généricité** — monter un foyer neuf ne demande que du paramétrage, jamais de développement.
2. **Cloisonnement** — aucun foyer ne peut voir les données d'un autre, même par erreur de code.

2. Décisions actées (2026-08-24)

#	Décision	Choix
D1	Mode de cloisonnement	Un projet Supabase par foyer (silo). Isolation physique, aucune colonne <code>foyer_id</code> , aucune donnée à migrer.
D2	Application Next.js	Un seul déploiement pour tous les foyers. La base cible est résolue à l'exécution , d'après le nom d'hôte.
D3	Appartenance	Un compte = un foyer . Pas de sélecteur de foyer, pas de table de jointure. Un même email peut exister dans deux bases : ce sont deux comptes distincts, sans lien.
D4	Routage	Sous-domaine par foyer — <code>ecoles.<domaine></code> , <code>foyerb.<domaine></code> . Résolu par le middleware avant connexion, donc l'écran de connexion porte déjà le bon logo.
D5	Vocabulaire	Revu le 2026-08-24 : inchangé. Un réglage de vocabulaire ne marche pas en français (accords) — voir §5.4. Réécriture neutre le jour où un foyer mixte se présente.
D6	Registre des foyers	Variable d'environnement JSON côté serveur. Ajouter un foyer = créer son projet Supabase + ajouter une entrée. Aucun redéploiement de code.

2.1. Ce que D1 ne résout pas

Le silo supprime la fuite **entre** foyers. Il ne supprime pas la fuite **à l'intérieur** d'un foyer.

Inventaire complet des politiques obtenu le 2026-08-24 (`supabase/audit-rls.sql`). Le diagnostic initial — « la plupart des tables n'ont pas de RLS » — était **faux**. La réalité est différente et plus subtile :

La RLS est active sur les 24 tables. La fuite vient de **16 politiques écrites** `T0 public USING (true)`. En PostgreSQL, `T0 public` ne veut pas dire « les personnes connectées » : il désigne **tous les rôles**, `anon` compris. Combiné au `GRANT ALL` que Supabase accorde par défaut à `anon` sur le schéma `public`, chacune de ces politiques ouvre la table à quiconque possède la clé publique.

Mesuré sans session, avec la seule clé `anon` :

Table	Lisible en anonyme	Cause
<code>residentes</code>	32 / 32 — nom, email, date de naissance, chambre, droits	policy <code>T0 public USING (true)</code>
<code>evenements</code>	88 / 88	idem
<code>meal_service_options</code>	429 / 429	idem
<code>invites_repas</code>	49 / 49	idem
<code>select_options_*</code>	60 / 60 (4 tables)	idem
<code>invites</code>	24 / 24	idem
<code>etages</code>	9 / 9	policy explicite <code>T0 anon</code>
<code>meal_options</code>	9 / 9	policy <code>T0 public</code>
<code>admin_sections</code>	4 / 4	idem
<code>residences</code>	3 / 3	policy explicite <code>T0 anon</code>
<code>app_settings</code>	3 / 3	policy <code>T0 public</code>
<code>invitees</code> , <code>pending_users</code>	1 / 1	idem — <code>pending_users</code> contient les emails en cours d'inscription
<code>places</code> , <code>groupes</code> , <code>groupe_membres</code> , <code>presences</code> , <code>absences_sejour</code> , <code>absences</code>	0	politiques visant <code>authenticated</code> 
<code>invitations</code> , <code>meal_audit_log</code>	0	RLS active, aucune policy — voir §2.3

2.2. **Faible critique — élévation de privilèges sur `residentes`**

La policy `UPDATE` nommée « Allow admins to update any residente » porte le prédicat :

```
EXISTS (SELECT 1 FROM residentes r WHERE r.user_id = auth.uid() OR r.is_admin = true)
```

Le `OR` est mal placé. Il suffit qu'**une seule ligne** de la table ait `is_admin = true` pour que le `EXISTS` soit vrai — il y en a **20**. Le prédicat ne dépend donc pas de l'appelant : **il est toujours vrai**. La policy vise `T0 public`, et `anon` détient `GRANT UPDATE`.

Conséquence : avec la seule clé publique, sans compte, on peut modifier n'importe quelle ligne de `residentes` — y compris s'attribuer `is_super_admin = true` ou `niveau_comptes = 3`, changer une

adresse email, ou vider les noms.

Non exploité à ce jour selon toute vraisemblance, mais exploitable depuis n'importe quel navigateur. **À corriger avant tout le reste** : `supabase/p0-hotfix-rls.sql`.

La correction est une simple suppression : aucune écriture sur `residentes` ne part du navigateur (15 accès client, tous en `.select`), tout passe par les routes `/api` sous `service_role`, qui possède `BYPASSRLS`.

2.3. Deux tables sans aucune policy

`invitations` et `meal_audit_log` ont la RLS **activée sans policy** : personne n'y accède, hors `service_role`. Elles fonctionnent uniquement parce que `createSupabaseServer()` tourne en service role. C'est la confirmation exacte du risque annoncé en §3.2 : le jour où ce client repasse en clé `anon`, ces deux tables cessent de répondre.

2.4. Le modèle de droits de la base est en retard sur celui du code

Une vingtaine de policies s'appuient sur le booléen `is_admin`, alors que l'application raisonne depuis 2026-08 en **5 sections × 4 niveaux** (`src/lib/roles.ts`). `is_admin` n'est plus lu nulle part dans le code — mais il est encore maintenu en base par le trigger `trg_residentes_sync_is_admin`, qui le calcule comme « a un accès admin quelconque ».

Les policies fonctionnent donc, mais **à la maille la plus grossière possible** : une personne « Admin · consulter Repas » y est indiscernable d'une « Admin · gérer Comptes ». C'est tout l'objet du durcissement §3.3–§3.4.

Autre incohérence relevée : `src/app/components/ConfirmationToggle.tsx:99` écrit directement dans `evenements`, dont la policy `UPDATE` exige `is_admin`. Une habitante qui confirme sa présence à un événement devrait donc être refusée — à vérifier en recette, la RPC `basculer_confirmation_evenement` (seule fonction `SECURITY DEFINER` de la base) semblait prévue pour ce cas.

2.5. Le correctif d'urgence

`supabase/p0-hotfix-rls.sql` — 30 instructions, sans risque applicatif :

1. supprime la policy `UPDATE` défaillante sur `residentes` (§2.2) ;
2. rebascule les 10 policies `SELECT` de `T0 public` vers `T0 authenticated` ;
3. supprime les lectures anonymes devenues inutiles (`pending_users`, `select_options_*` héritées, doublon sur `residences`) ;
4. supprime les 4 policies d'écriture `USING (true)` sur `invites` / `invites_repas`.

`residences` et `etages` **restent lisibles en anonyme** : le formulaire d'inscription des invitées (`signupForm.tsx` via `useResidences`) en a besoin avant création du compte.

Ce correctif ferme la fuite **anonyme**. Il ne corrige pas la sur-exposition **entre personnes connectées** — une résidente lit toujours l'email et la date de naissance de toutes les autres. C'est l'objet de P0 complet.

3. P0 — Fermer les RLS

Préalable à tout. À livrer même si le second foyer était abandonné.

3.1. Pourquoi ça ne peut pas se régler côté application

58 requêtes `.from(...)` partent directement du navigateur (contre 140 dans les routes API). Elles ne traversent aucune garde serveur. Rien d'autre que la RLS ne peut les protéger.

3.2. Corriger d'abord `createSupabaseServer`

`src/lib/supabaseServer.ts:15` passe `SUPABASE_SERVICE_ROLE_KEY` au client dit « de session ». Deux conséquences :

- session absente ou expirée → la requête part en service role, RLS contournée ;
- toute policy écrite aujourd'hui serait ignorée par ce client, donc invérifiable.

Il faut y passer `NEXT_PUBLIC_SUPABASE_ANON_KEY`.

⚠ **C'est un changement à risque.** Une quinzaine d'appels utilisent aujourd'hui `createSupabaseServer()` en comptant *implicitement* sur le service role. Le cas le plus net : `src/app/api/admin/users/route.ts:20` liste **toutes** les `residentes`. Sous RLS réelle, cette lecture échouera si aucune policy n'autorise une admin « Comptes » à voir tout le monde. Il faut donc auditer les 15 sites d'appel un par un et trancher pour chacun :

- donnée propre à l'appelante → client session (RLS), policy `user_id = auth.uid()` ;
- donnée de tout le foyer → passer par `requireSectionView` / `requireSectionEdit`, qui renvoient déjà le client service role après contrôle des droits.

Les 7 routes sans garde (`check-user`, `invite-repas`, `sync-user`, `invites`, `get-is-absent`, `activation/complete`, `admin/users`) sont à reprendre dans le même mouvement : certaines sont légitimement publiques (`check-user`, `activation/complete`), les autres non.

3.3. Fonctions d'appui

Pour éviter d'écrire la même sous-requête dans chaque policy, et pour ne pas déclencher de récursion sur `residentes` :

```
-- SECURITY DEFINER : contourne la RLS de residentes, sinon la policy
-- de residentes s'appellerait elle-même.
create or replace function public.mon_niveau(section text)
returns smallint language sql stable security definer set search_path = public as $$
select case
  when r.is_super_admin or r.is_technique then 3
  else coalesce(
```

```

    case section
      when 'repas' then r.niveau_repas
      when 'evenements' then r.niveau_evenements
      when 'absences' then r.niveau_absences
      when 'comptes' then r.niveau_comptes
      when 'infos' then r.niveau_infos
    end, 1)
  end
  from public.residentes r where r.user_id = auth.uid();
$$;

```

Le vocabulaire est celui de `src/lib/roles.ts` : 0 Masquée · 1 Habitante · 2 Admin consulter · 3 Admin gérer. Les politiques rejouent donc exactement `canAccessSection` / `canViewSection` / `canEditSection`, côté base cette fois.

3.4. Tableau des politiques cibles

Table	Lecture	Écriture
residentes	soi-même ou <code>mon_niveau('comptes') >= 2</code>	soi-même (champs profil) ; droits et statut → <code>>= 3</code>
presences	soi-même ou <code>mon_niveau('repas') >= 2</code>	soi-même ou <code>>= 3</code>
absences_sejour	soi-même ou <code>mon_niveau('absences') >= 2</code>	soi-même ou <code>>= 3</code>
evenements	<code>mon_niveau('evenements') >= 1</code>	<code>>= 3</code>
invites_repas, invites, invitees	<code>mon_niveau('repas') >= 1</code> (les siennes) ou <code>>= 2</code>	siennes, ou <code>>= 3</code>
admin_sections	<code>mon_niveau('infos') >= 1</code>	<code>>= 3</code>
meal_options, meal_service_options	authenticated	<code>mon_niveau('repas') >= 3</code>
app_settings	authenticated	<code>mon_niveau('repas') >= 3</code> (verrous repas)
residences, etages, places	authenticated	<code>mon_niveau('comptes') >= 3</code>
groupes, groupe_membres	déjà en place	à compléter en écriture
pending_users	aucune (service role uniquement)	idem
invitations	<code>mon_niveau('comptes') >= 2</code>	<code>>= 3</code>
meal_audit_log	<code>mon_niveau('repas') >= 2</code>	insertion service role
select_options_*	(tables supprimées en P3)	—

Remarques :

- `residences` et `etages` doivent **rester lisibles en anonyme** — le formulaire d'inscription des invitées les lit avant création du compte. Ne pas les fermer sans avoir d'abord déplacé cette lecture côté serveur.
- Les politiques existantes s'appuient sur le booléen `is_admin` (§2.4) : chacune est à **réécrire**, pas à ajouter. Une fois `mon_niveau()` en place et toutes les politiques migrées, `is_admin` et son trigger `trg_residentes_sync_is_admin` pourront être supprimés.
- `invitations` et `meal_audit_log` n'ont **aucune** policy (§2.3) : elles en ont besoin avant le passage de `createSupabaseServer` en clé anon.

3.5. Recette

Un script de non-régression, à garder et à rejouer avant chaque mise en production :

1. avec la clé anon **sans session**, `select` sur les 24 tables → **0 ligne partout** ;
2. connecté en simple habitante → ne voit que ses propres `presences` / `absences_sejour` ;
3. connecté en « Admin consulter Repas » → voit toutes les présences, **ne peut pas** écrire ;
4. `update` sur `app_settings` par une habitante → refusé.

4. P1 — Un socle de base reproductible fait le 2026-08-24

Avant : **on ne pouvait pas recréer la base à partir du dépôt.**

Les 23 fichiers `supabase/*.sql` étaient des **patches successifs**, pas un socle. Ils créaient 6 tables sur 24 (`places`, `invitations`, `etages`, `groupes`, `groupe_membres`, `meal_audit_log`). Les 18 autres — `residentes`, `presences`, `evenements`, `absences_sejour`, `app_settings`, `admin_sections`... — n'existaient que dans l'interface Supabase.

4.1. Ce qui a été fait

`supabase/migrations/20260824000000_socle.sql` — schéma complet extrait par `pg_dump --schema-only --schema=public --no-owner`, après P0 :

tables	24
politiques	50
RLS active	24 / 24
fonctions	3 (<code>mon_niveau</code> , <code>basculer_confirmation_evenement</code> , <code>residentes_sync_is_admin</code>)
triggers	1
index	16
données	aucune

valeurs propres à un foyer	aucune — vérifié : plus aucune occurrence de '12', '36' ou 'corail'
----------------------------	--

Ce dernier point confirme que `blocs-dynamiques.sql:57` avait bien levé la contrainte `check (residence in ('12','36','corail'))` de `places`. C'était une inconnue tant qu'on n'avait pas le schéma réel.

Le CLI Supabase n'a **pas** été utilisé : `supabase db pull` réclame Docker, absent de la machine. `pg_dump` en direct fait le même travail sans intermédiaire. Il a fallu des outils client PostgreSQL 17+ (`brew install libpq`), le serveur Supabase tournant en 17.6 — `pg_dump` refuse de lire un serveur plus récent que lui.

Deux retouches sur la sortie brute :

- suppression des méta-commandes `\restrict / \unrestrict` (nouveau de `pg_dump 18`) : elles ne fonctionnent que dans `psql` et font échouer le script dans l'éditeur SQL de Supabase ;
- `CREATE SCHEMA public` → `CREATE SCHEMA IF NOT EXISTS public` : le schéma existe déjà sur tout projet neuf.

supabase/seed.sql — contenu d'un foyer vierge, idempotent : les 3 verrous d' `app_settings`, les 4 rubriques Administratif créées vides. Volontairement **sans** blocs, étages, chambres, groupes ni options de repas : cette saisie est le paramétrage du foyer, elle appartient à l'intendance.

scripts/foyer-nouveau.mjs — amorçage : crée le compte technique (super-admin, sans place, jamais compté dans la capacité) et sa ligne `residentes`. Seule étape impossible en SQL, un mot de passe valide devant passer par l'API d'authentification. Le script vérifie que socle et seed sont en place, refuse de se rejouer si un compte technique existe, et n'affiche le mot de passe qu'une fois.

supabase/migrations/archive/ — les 23 patches, plus un `LISEZ-MOI.md`. Ils gardent leur valeur documentaire : leurs commentaires expliquent le *pourquoi* de chaque décision de modèle, ce qu'un dump ne dit pas. Deux d'entre eux (`lot3-migrate-residentes.sql`, `lot3-seed-chambres.sql`) sont des migrations de données à sens unique propres au Foyer des Écoles — à ne jamais rejouer ailleurs.

4.2. Socle régénéré le 2026-08-24

Le tableau ci-dessus décrit le **premier** socle, extrait avant P2. Une fois P2 et P3 passés sur les deux foyers, il a été régénéré depuis la base des Écoles et absorbe désormais toutes les migrations `p2*` / `p3*` :

	premier socle	socle courant
fichier	<code>20260824000000_socle.sql</code>	<code>20260824120000_socle.sql</code>
tables	24	22 (deux tables héritées supprimées par P3)
polices	50	49
fonctions	3	4 (<code>est_super_admin</code> ajoutée par P2b)

Monter un foyer neuf demande donc **trois** fichiers, et non plus huit : `socle` · `storage-branding.sql` · `seed.sql`.

Le bucket a son propre fichier pour une raison de fond : il vit dans le schéma `storage`, que `pg_dump --schema=public` ne capture pas. Une régénération ne le contiendra donc jamais, et son absence ne lève aucune erreur — seulement un téléversement de logo qui échoue, des semaines plus tard.

4.3. Recette de P1 — faite

Partir d'un **projet Supabase vide**, puis :

1. jouer `supabase/migrations/20260824000000_socle.sql` ;
2. jouer `supabase/seed.sql` ;
3. `FOYER_URL=... FOYER_SERVICE_KEY=... FOYER_ADMIN_EMAIL=... node scripts/foyer-nouveau.mjs` ;
4. pointer l'application sur ce projet et vérifier qu'on peut se connecter, créer un bloc, un étage, une chambre, et inviter quelqu'un — **sans toucher au code**.

Ce test ne peut pas se faire sur la base existante : il exige une base réellement vide. C'est aussi la première brique du second foyer, donc le projet créé n'est pas jetable.

5. P2 — Dé-brandeur fait le 2026-08-24

5.1. Ce qui est sorti du code

Avant	Maintenant
<code>layout.tsx</code> — <code>title: 'Les Écoles', description</code>	<code>generateMetadata()</code> lit <code>foyer_nom</code> / <code>foyer_description</code>
<code>public/manifest.json</code> figé sur « Foyer des Écoles »	supprimé → <code>src/app/manifest.ts</code> , engendré à la demande
<code>theme_color: '#004AAD'</code> en dur	<code>generateViewport()</code> lit <code>foyer_couleur</code>
<code>signin/page.tsx</code> — <code>/logo.png</code> , <code>alt="Logo des écoles"</code>	<code>foyer_logo_url</code> , ou le nom du foyer en toutes lettres
<code>Europe/Paris</code> dans <code>lockUtils</code> et <code>foyerLock</code>	paramètre <code>fuseau</code> , câblé jusqu'aux 5 appelants
<code>toLocaleDateString("fr-FR")</code>	paramètre <code>locale</code>

5.2. Les réglages

`supabase/p2-identite-foyer.sql` — idempotent, aussi intégré à `seed.sql` :

Clé	Défaut	Usage
<code>foyer_nom</code>	Foyer	titre de l'onglet, manifeste, emails
<code>foyer_nom_court</code>	Foyer	<code>short_name</code> de l'application installée

Clé	Défaut	Usage
foyer_description	...	métadonnées, manifeste
foyer_couleur	#004AAD	barre du navigateur, écran d'accueil
foyer_logo_url	(vide)	écran de connexion et icônes ; vide → le nom s'affiche en toutes lettres
foyer_fuseau	Europe/Paris	heure de référence des verrous
foyer_locale	fr-FR	format des dates

Les valeurs par défaut sont **neutres** : un foyer neuf s'affiche correctement avant même d'être personnalisé, et aucun nom réel n'est écrit dans le dépôt.

5.3. Trois décisions d'implémentation

- **Lecture publique des seules clés foyer_***. L'écran de connexion et le manifeste s'affichent avant toute session ; les heures de verrouillage restent réservées aux comptes. La policy utilise `starts_with(key, 'foyer_')` et non `like 'foyer_%'` — dans un `LIKE`, le tiret bas est un joker et `foyerX` passerait.
- **L'identité descend du layout serveur vers les composants client** par un contexte (`useIdentite`), plutôt que d'être rechargée depuis le navigateur : sinon l'écran de connexion afficherait un vide le temps d'un aller-retour. `identiteFoyer()` est mémorisé par requête via `cache()` de React.
- ** et non next/image** pour le logo : `next/image` exige de déclarer chaque domaine dans `next.config.ts`, impraticable avec un projet Supabase par foyer.

5.4. Décision D5 revue — le vocabulaire reste inchangé

La décision initiale prévoyait des réglages `mot_resident_singulier` / `_pluriel`. **Abandonné le 2026-08-24**, après mesure : 130 occurrences de vocabulaire féminin sur 22 fichiers, et surtout le français **accorde**. « Aucune résidente inscrite » ne devient pas « Aucun résident inscrit » en substituant un mot : il faut l'article, l'adjectif et le participe. Un réglage de vocabulaire produirait du français cassé.

Les deux foyers visés étant des foyers d'étudiantes, le sujet est **reporté**. Le jour où un foyer mixte se présentera, le bon geste sera une **réécriture neutre** des chaînes (« les membres », « les comptes », « personne d'inscrit »), sans réglage — et non deux jeux de libellés, qui imposeraient d'écrire chaque phrase en plusieurs versions pour toujours.

5.5. P2b — L'identité se règle depuis l'application

Décidé le 2026-08-24 : plutôt que d'écrire les valeurs de chaque foyer par script, **le super-admin du foyer les règle lui-même**, logo compris. Un foyer qui change de nom ou de logo n'a besoin de personne.

Écran — Administration → Identité du foyer, en tête de la page Administration. Le composant se retire de lui-même si la personne n'est pas super-admin.

Trois gardes, pas une :

Niveau	Mécanisme
Affichage	<code>IdentiteFoyerSettings</code> ne rend rien hors super-admin
API	<code>requireSuperAdmin()</code> sur <code>/api/admin/identite</code> et <code>/api/admin/identite/logo</code>
Base	policy <code>app_settings: ecriture gestion</code> — les clés <code>foyer_*</code> exigent <code>est_super_admin()</code>

La troisième comble un trou : la policy précédente ne regardait que le **niveau**, jamais la **clé** touchée. Une « Admin · gérer Repas » aurait pu renommer le foyer. `mon_niveau()` ne pouvait pas servir ici — elle renvoie 3 aussi bien pour un super-admin que pour un « Admin · gérer » — d'où la nouvelle fonction `est_super_admin()`.

Logo — bucket Supabase Storage `branding`, **public en lecture** puisque le logo s'affiche avant toute connexion, donc sans jeton. L'écriture ne passe pas par une policy de storage mais par la route API sous service role : une seule porte d'entrée, un seul endroit où le droit se vérifie. Le fichier est horodaté à chaque téléversement, sans quoi les navigateurs continueraient d'afficher l'ancien logo depuis leur cache.

Validations côté serveur : liste blanche des clés (un appel direct ne peut pas écrire les verrous), nom non vide, couleur au format `#RRGGBB`, et le fuseau est éprouvé par un `toLocaleString` — un fuseau erroné casserait tous les calculs de verrouillage sans message clair.

5.6. Un piège corrigé : le titre figé au build

`generateMetadata` lit le nom du foyer en base, mais Next pré-rendait `/homePage`, `/calendrier` et `/signin` **au moment du build** : le titre y était gravé une fois pour toutes, et renommer un foyer n'aurait rien changé. `export const dynamic = "force-dynamic"` dans le layout racine règle le cas pour tout l'arbre — l'application étant entièrement derrière authentification, le rendu statique n'apportait rien. Vérifié : les 17 routes sont désormais marquées `f`.

C'est le piège annoncé en §7.3 pour P4, rencontré plus tôt que prévu.

5.7. P2c — Amorcer un foyer côté client

Au démarrage d'un foyer, ni bloc, ni étage, ni chambre : la voie d'invitation ordinaire, qui exige une place libre, ne peut pas servir. C'est pourtant le moment où il faut passer la main, pour que l'installation ne dépende plus du compte technique.

`invitations.place_id` devient facultatif. **Une invitation sans place vaut invitation de super-administratrice** : elle n'occupe aucune chambre et n'entre pas dans la capacité, comme le compte technique. Deux contraintes garantissent la cohérence — `super_admin` n'a jamais de place, `residente` en a toujours une.

Le panneau d'invitation est **réservé au compte technique** (`requireTechnique`) : nommer une super-administratrice relève de l'installation, pas de l'administration courante. Un foyer ne se donne pas lui-même de nouveaux détenteurs des pleins droits.

Se logger ensuite. Une super-administratrice sans chambre voit en tête de l'écran Administration un encadré « Choisir ma chambre », limité à son propre compte et qui disparaît dès qu'elle est logée. Un premier essai avait ouvert le panneau de maintenance « Sans chambre attribuée » à toute la gestion des comptes : mauvaise porte, ce panneau signale des anomalies techniques et reste réservé au compte technique.

5.8. P2d — L'icône n'est pas le logo

Le manifeste réutilisait `foyer_logo_url` comme icône d'écran d'accueil. Deux défauts, constatés sur téléphone :

- un logo d'en-tête est presque toujours **transparent** ; iOS et Android composent la transparence sur du **noir** ;
- un logo est **large** — celui des Écoles fait 2,6:1 — et son texte devient illisible comprimé dans un carré de 180 px.

D'où un réglage distinct `foyer_icone_url`, son propre emplacement de téléversement, et un manifeste qui ne retombe plus sur le logo mais sur une icône neutre. Pour Les Écoles, l'icône a été réduite à la marque (les quatre livres) sur fond blanc opaque : 25 Ko contre 977 Ko pour le logo d'origine, lui-même ramené à 39 Ko.

5.9. P2e — Retrait du réglage de couleur

`foyer_couleur` n'alimentait que le `theme_color` du manifeste : barre du navigateur sur Android, écran de démarrage de l'application installée. Boutons, titres et bandeaux sont des classes Tailwind écrites en dur dans une centaine d'endroits — passer le réglage en rouge laissait l'application bleue.

Un réglage qui promet un thème sans en livrer un déroute plus qu'il n'aide : il est retiré, la teinte devient la constante `COULEUR_APPLI`. Elle redeviendra un réglage le jour où l'interface passera à des variables CSS — chantier non planifié.

5.10. Reste à faire

- Les dates affichées utilisent encore `"fr-FR"` en dur (9 fichiers). Sans effet tant que tous les foyers sont francophones ; à reprendre avec `foyer_locale` au premier foyer qui ne l'est pas.
- Thématiser l'interface (variables CSS) si la couleur doit un jour piloter autre chose que la barre du navigateur.

6. P3 — Purger la dette qui empêche la généricité

À faire **avant** de dupliquer le modèle, sinon on duplique la dette.

6.1. `select_options_residence` — double source de vérité

Table héritée, encore lue par 3 écrans en parallèle de `places` / `etages` :

- `src/app/profil/page.tsx:78`
- `src/app/admin/foyer/page.tsx:80`
- `src/app/admin/repas/page.tsx:87`

Deux endroits déclarent les mêmes chambres. Un foyer neuf devrait les saisir deux fois. → Basculer les 3 écrans sur `places / etages`, puis supprimer la table et ses sœurs inutilisées (`select_options_evenement`, `select_options_rappel`, `select_options_recurrence`), ainsi que la table `absences` (remplacée par `absences_sejour`).

6.2. Types figés sur les blocs actuels

- `src/types/InviteRepas.ts:6` — `lieu_repas: "12" | "36"`. Un type littéral sur les blocs du foyer actuel. → `lieu_repas: string`.
- `src/lib/residences.ts:59-61` et `:70` — `couleurs` et `kind` de repli pour 12 / 36 / `corail`. Ces replis servaient à survivre avant `blocs-dynamiques.sql`, qui est passé. → Supprimer `couleurDefault`, garder l'attribution cyclique par index.

6.3. Fuseau et locale

41 occurrences de `Europe/Paris` / `fr-FR`. Le calcul de verrou (`src/lib/foyerLock.ts:47`, `src/lib/lockUtils.ts`) raisonne explicitement à l'heure de Paris — c'est **juste** aujourd'hui, et faux pour un foyer ailleurs. → Faire descendre `foyer_fuseau` et `foyer_locale` depuis le contexte, avec `Europe/Paris` / `fr-FR` en valeur par défaut.

Priorité basse si les deux foyers sont en France : à traiter au premier foyer hors métropole.

6.4. Self-signup des invitées

`src/app/components/signupForm.tsx` permet à une « invitée » de créer son compte librement. Avec le silo, ça reste cohérent : le sous-domaine détermine la base, donc le foyer. Rien à changer — mais à **vérifier en recette** que le formulaire atteint bien la base du sous-domaine et pas une autre.

6.5. Ce que P3 avait mal jugé — les listes d'options d'événement

`p3-nettoyage.sql` conservait `select_options_evenement` et `select_options_rappel` au motif qu'elles étaient « des listes de configuration propres à chaque foyer, comme les options de repas ». **C'était faux**, et le second foyer l'a montré avant nous.

Ces tables étaient remplies **à la main** sur le premier foyer, jamais par le seed. Un foyer neuf en héritait **vides** : aucun type d'événement à choisir, et comme la catégorie est obligatoire à la validation (`AjoutEventModal.tsx:107`), **aucun événement créable**. Trois symptômes rapportés par la cliente — plus de type, plus de rappel, création impossible — pour un seul défaut.

L'erreur de raisonnement mérite d'être nommée : j'avais comparé ces tables aux options de repas **sans vérifier qu'un écran les édite**. Les options de repas en ont un ; celles-ci, non. Une table que personne ne peut modifier n'offre aucune souplesse — seulement une occasion de l'oublier au moment du seed.

Deux raisons de plus de les ramener dans le code (`src/lib/evenementOptions.ts`) :

- la valeur `intendance` **porte un comportement** : les confirmations d'un événement de cette catégorie se lisent « Fait » et non « Je participe » (`ConfirmationToggle.tsx`). Une liste librement modifiable pouvait casser cette fonction en silence ;
- un délai de rappel n'a rien de propre à un foyer.

Les `value` sont inchangées : l'historique de `evenements.category` reste lisible. `supabase/p4-options-evenement-en-code.sql` supprime les deux tables.

Le contrôle qui manquait, et qui vaut pour la suite : après avoir monté un foyer neuf, ne pas se contenter de vérifier que les écrans s'affichent — **essayer d'y créer quelque chose**. Un formulaire dont une liste obligatoire est vide s'affiche parfaitement.

7. P4 — Résolution du foyer à l'exécution fait le 2026-08-24

Avant, l'URL et les clés Supabase venaient de `process.env`, et `NEXT_PUBLIC_SUPABASE_URL` était **figée dans le bundle au moment du build**. Un déploiement ne pouvait donc servir qu'un foyer.

7.1. Le registre

Variable d'environnement **serveur** `Foyers`, un tableau JSON. Jamais `NEXT_PUBLIC_` : elle contient les clés service role.

```
[
  { "slug": "ecoles", "host": "ecoles.exemple.fr",
    "url": "https://aaa.supabase.co", "anon": "...", "serviceRole": "..." },
  { "slug": "guerledan", "host": "guerledan.exemple.fr",
    "url": "https://bbb.supabase.co", "anon": "...", "serviceRole": "..." }
]
```

Ajouter un foyer ne demande **aucune modification de code**, mais bien un **redéploiement** : Next fige les `process.env` du middleware au build. La formulation initiale du plan (« aucun redéploiement ») était inexacte.

7.2. Deux modules, et pourquoi

Fichier	Rôle	Contrainte
<code>src/lib/foyers.ts</code>	registre, <code>foyerParHost(host)</code>	pur — importé par le middleware, qui tourne en Edge où <code>next/headers</code> n'existe pas
<code>src/lib/foyerServeur.ts</code>	<code>foyerCourant()</code> , <code>identiteFoyer()</code>	serveur uniquement

Fichier	Rôle	Contrainte
<code>src/lib/foyer.ts</code>	types, valeurs par défaut, parseur, listes	pur — <code>lockUtils</code> et <code>foyerLock</code> y prennent <code>IDENTITE_DEFAULT</code> et servent dans des composants navigateur

Cette séparation n'est pas cosmétique : les deux premières tentatives ont échoué au build parce qu'un import de `next/headers` remontait, d'abord jusqu'au middleware, ensuite jusqu'au bundle navigateur par la chaîne `AbsenceModal` → `foyerLock` → `foyer`.

7.3. Ce qui a été recâblé

Fichier	Changement
<code>src/lib/supabaseServer.ts</code>	les deux clients visent la base du foyer courant ; <code>createSupabaseAdmin()</code> devient asynchrone (6 sites d'appel passés en <code>await</code>)
<code>src/middleware.ts</code>	résout le foyer depuis <code>request.headers.get("host")</code>
<code>src/app/providers.tsx</code>	reçoit <code>supabaseUrl</code> / <code>supabaseAnonKey</code> en props au lieu de lire <code>process.env</code>
<code>src/app/layout.tsx</code>	résout le foyer et les transmet
<code>src/lib/foyerServeur.ts</code>	<code>identiteFoyer()</code> lit la base du foyer courant

7.4. Repli mono-foyer

Sans `F0YERS`, le registre se rabat sur `NEXT_PUBLIC_SUPABASE_*` et fabrique une entrée unique. Le développement local et un déploiement mono-foyer continuent donc de fonctionner sans registre — c'est ce qui permet de livrer P4 sans rien casser.

En développement, l'hôte vaut `localhost:3000` et ne correspond à rien : on prend `F0YER_DEV` s'il est défini, sinon la première entrée. Un hôte reconnu l'emporte toujours sur `F0YER_DEV`.

Un `F0YERS` mal formé n'est **pas** ignoré en silence — il servirait le mauvais foyer à tout le monde. Le registre journalise l'erreur avant de se replier.

7.5. Vérifié

Résolution éprouvée hors application (registre transpilé, exécuté sur des cas réels) : correspondance exacte, insensible à la casse, port ignoré, repli, `F0YER_DEV`, normalisation de l'URL. Build et lint verts, 0 erreur.

7.6. Points de vigilance

- **Cookies.** Supabase nomme son cookie `sb-<ref-projet>-auth-token` : deux projets, deux noms. Ajouté aux sous-domaines distincts, aucune session ne peut se mélanger. Ne **jamais** poser le cookie sur le domaine parent (`.exemple.fr`), ce qui les remélangerait.
- **Redirections d'authentification.** Chaque projet Supabase a sa propre *Site URL* et sa propre liste d'URL de redirection : y renseigner le sous-domaine du foyer. `/api/admin/invitations` utilise

`req.nextUrl.origin`, ce qui est déjà correct.

- **Rendu statique.** Réglé dès P2 par `export const dynamic = "force-dynamic"` dans le layout racine ; sans lui, le foyer aurait été résolu une fois pour toutes au build.
- **Domaine générique.** DNS et Vercel en `*.exemple.fr`, pour ajouter un foyer sans toucher à la configuration réseau.

7.7. Deux fuites du câblage, trouvées à l'usage

Le passage au multi-foyer consistait à remplacer chaque lecture de `process.env` par une résolution d'après le nom d'hôte. Deux endroits y ont échappé, et aucun n'appelait `createSupabaseServer` ni `createSupabaseAdmin` — c'est ce qui les a rendus invisibles à l'audit des sites d'appel.

`src/app/auth/confirm/route.ts` construisait son propre client : elle n'a pas encore de session à lire quand elle vérifie un jeton d'email. Conséquence : tout jeton émis par un autre foyer que celui par défaut était vérifié contre la mauvaise base, et l'invitée lisait « Lien invalide ou expiré » alors que son lien était parfaitement valide. Corrigé, et les échecs journalisent désormais l'hôte, le type et le motif — un « lien expiré » sans trace était indistinguishable.

Leçon de méthode : auditer les appelants d'une fonction ne suffit pas quand le motif à traquer est l'usage d'une *variable d'environnement*. Le contrôle juste est `grep -rn "NEXT_PUBLIC_SUPABASE" src`, qui ne doit plus laisser que `src/lib/foyers.ts` (le repli mono-foyer, intentionnel).

7.8. Le gabarit d'email, pièce adjacent

Le symptôme « lien expiré » avait une **seconde** cause, indépendante du code : le gabarit d'email **par défaut** de Supabase pointe vers son propre `/auth/v1/verify`. Supabase vérifie le jeton lui-même, le **consomme**, puis redirige avec un code que l'application ne sait pas traiter. L'invitée voit « lien expiré », et le jeton est brûlé.

L'application attend `{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=invite&next=/activation`. Le gabarit étant un réglage **du projet Supabase**, un foyer neuf part toujours sur le défaut : voir `supabase/GABARITS-EMAIL.md`, qui donne aussi le tableau symptôme → cause et les règles de partage d'un même SMTP entre foyers (le `Sender name` doit être propre à chaque foyer).

8. P5 — Exploitation

8.1. Rejouer les migrations sur N bases

C'est le coût assumé de D1. À automatiser dès le deuxième foyer, sinon les bases divergent :

`scripts/migrate-tous.mjs` — lit `F0YERS`, et pour chacun lance `supabase db push --db-url postgres://postgres.<ref>:<mdp>@...pooler.supabase.com:5432/postgres`.

Prévoir un mode `--dry-run` et **s'arrêter à la première erreur** : une base à moitié migrée est pire que deux versions différentes.

8.2. Recette avant chaque mise en production

1. le script RLS de §3.5 passe sur **chaque** base ;
2. `ecoles.exemple.fr` et `foyerb.exemple.fr` affichent chacun leur logo **avant** connexion ;
3. connexion sur A, puis navigation vers B → non connecté (et non : connecté sur les données de A) ;
4. un compte de A ne peut pas se connecter sur B.

9. Ordre et charge

Phase	Contenu	Bloquant pour	Charge
P0	RLS + clé anon dans <code>createSupabaseServer</code>	✅ fait	
P1	Socle <code>supabase/migrations/</code> + <code>seed.sql</code> + amorçage	✅ fait	
P2	Dé-branding, identité réglable en application (P2b→P2e)	✅ fait	
P3	<code>select_options_residence</code> , types figés, locale des dates	reste	
P4	Résolution du foyer par hôte	✅ fait	
P5	Migrations multi-bases, recette	partiel — <code>verif-rls.mjs</code> et <code>verif-foyers.mjs</code> livrés	

P2 et P3 sont indépendantes et peuvent s'intercaler. **P0 d'abord, P1 ensuite** : sans socle reproductible, la RLS du second foyer serait re-saisie à la main, donc différente.